

6. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

OUTPUT :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define SERVER_FIFO "/tmp/server_fifo"
#define BUFFER_SIZE 512

int main() {
    char buffer[BUFFER_SIZE];

    // Create server FIFO
    mkfifo(SERVER_FIFO, 0666);

    printf("Server started...\n");
    printf("Waiting for clients...\n");

    // Open server FIFO
    int fd = open(SERVER_FIFO, O_RDONLY);

    if (fd == -1) {
        perror("open");
        return 1;
    }

    while (1) {
        memset(buffer, 0, sizeof(buffer));

        // Read message from client
        ssize_t n = read(fd, buffer, sizeof(buffer) - 1);

        if (n <= 0) {
            continue;
        }

        buffer[n] = '\0';

        /*
         * Message format:
         * PID:message
         */
        char *colon = strchr(buffer, ':');

        if (colon == NULL) {
            printf("Invalid message\n");
            continue;
        }

        *colon = '\0';

        char *pid = buffer;
        char *message = colon + 1;

        printf("Client %s: %s\n", pid, message);

        // Create client's response FIFO
        char response_fifo[100];

        snprintf(response_fifo,
                 sizeof(response_fifo),
                 "/tmp/client_%s_fifo",
                 pid);

        // Send response
        int response_fd = open(response_fifo, O_WRONLY);

        if (response_fd == -1) {
            perror("response FIFO");
            continue;
        }

        char response[BUFFER_SIZE];

        snprintf(response,
                 sizeof(response),
                 "Server received your message: %s",
                 message);

        write(response_fd, response, strlen(response));

        close(response_fd);
    }

    close(fd);
    unlink(SERVER_FIFO);

    return 0;
}
```

```
abhinav@Abhinav:~/OS$ nano practical6A.c
abhinav@Abhinav:~/OS$ gcc practical6A.c -o practical6A
abhinav@Abhinav:~/OS$ ./practical6A
Server started...
Waiting for clients...
```

6(B) :

```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define SERVER_FIFO "/tmp/server_fifo"
#define BUFFER_SIZE 512

int main() {
    char message[BUFFER_SIZE];

    // Get process ID
    pid_t pid = getpid();

    // Create unique FIFO for this client
    char client_fifo[100];

    snprintf(client_fifo,
             sizeof(client_fifo),
             "/tmp/client_%d_fifo",
             pid);

    mkfifo(client_fifo, 0666);

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);

    message[strcspn(message, "\n")] = '\0';

    // Open server FIFO
    int server_fd = open(SERVER_FIFO, O_WRONLY);

    if (server_fd == -1) {
        perror("server FIFO");
        unlink(client_fifo);
        return 1;
    }

    // Create message: PID:message
    char buffer[BUFFER_SIZE];

    snprintf(buffer,
             sizeof(buffer),
             "%d:%s",
             pid,
             message);

    // Send message
    write(server_fd, buffer, strlen(buffer));

    close(server_fd);

    // Wait for server response
    int client_fd = open(client_fifo, O_RDONLY);

    if (client_fd == -1) {
        perror("client FIFO");
        unlink(client_fifo);
        return 1;
    }

    memset(buffer, 0, sizeof(buffer));

    ssize_t n = read(client_fd, buffer, sizeof(buffer) - 1);

    if (n > 0) {
        buffer[n] = '\0';
        printf("Server: %s\n", buffer);
    }

    close(client_fd);

    // Remove client FIFO
    unlink(client_fifo);

    return 0;
}
```

```
abhinav@Abhinav:~/OSS$ nano practical6B.c
abhinav@Abhinav:~/OSS$ gcc practical6B.c -o practical6B
abhinav@Abhinav:~/OSS$ ./practical6B
Enter message: hello
```

6(C) :

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

volatile sig_atomic_t running = 1;
volatile sig_atomic_t usr1_received = 0;

void handle_signal(int signal) {
    if (signal == SIGINT) {
        const char msg[] = "\nSIGINT received\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);

        running = 0;
    }

    else if (signal == SIGTERM) {
        const char msg[] = "\nSIGTERM received\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);

        running = 0;
    }

    else if (signal == SIGUSR1) {
        const char msg[] = "\nSIGUSR1 received\n";
        write(STDOUT_FILENO, msg, sizeof(msg) - 1);

        usr1_received = 1;
    }
}

int main() {
    struct sigaction action;

    action.sa_handler = handle_signal;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    // Register SIGINT
    sigaction(SIGINT, &action, NULL);

    // Register SIGTERM
    sigaction(SIGTERM, &action, NULL);

    // Register SIGUSR1
    sigaction(SIGUSR1, &action, NULL);

    printf("Signal handling program started\n");
    printf("PID: %d\n", getpid());

    printf("Send SIGUSR1 using:\n");
    printf("kill -SIGUSR1 %d\n", getpid());

    printf("Press Ctrl+C for SIGINT\n");

    while (running) {
        printf("Program is running...\n");

        sleep(2);

        if (usr1_received) {
            printf("Main program processed SIGUSR1 event\n");

            usr1_received = 0;
        }
    }

    printf("Program terminating...\n");

    return 0;
}
```

```
abhinav@Abhinav:~/OSS$ nano practical6C.c
abhinav@Abhinav:~/OSS$ gcc practical6C.c -o practic
abhinav@Abhinav:~/OSS$ ./practical6C
Signal handling program started
PID: 868
Send SIGUSR1 using:
kill -SIGUSR1 868
Press Ctrl+C for SIGINT
Program is running...
```

S